

Beginner Roadmap — Hyperlocal Climate Digital Twin (Western Maharashtra)

This is a from-zero, chunk-by-chunk plan. Each chunk has: **what to do**, **what to download** (with source + rough size), and a **checkpoint** you must hit before moving on. Don't skip checkpoints — they're what stop the project from turning into a folder of half-finished datasets.

Assumption: you have a laptop (i5/i7-class CPU, 16GB RAM, entry GPU) and are comfortable installing software but not necessarily experienced with GIS/ML.

PHASE 0 — Environment Setup (Days 1-3)

Goal: a working Python + GIS + ML environment. Nothing scientific happens yet.

Chunk 0.1 — Core install

- Install **Python 3.11** (via python.org or `conda`/`miniconda`) — miniconda is easier for geospatial packages).
- Install **Git** and create a GitHub repo for the project.
- Install **VS Code** (or your editor of choice) + Python/Jupyter extensions.

Chunk 0.2 — Environment

Create a conda environment and install:

```
conda create -n climate python=3.11
conda activate climate
conda install -c conda-forge geopandas rasterio gdal xarray netCDF4 cfgrib
rioxarray
pip install pandas numpy scikit-learn xgboost lightgbm torch torch-
geometric matplotlib jupyterlab
```

- Install **QGIS** (free desktop GIS app) from qgis.org — you'll use this to *look at* your data, not just process it in code. This matters a lot for a beginner: seeing your grid/DEM visually catches mistakes code won't.

Chunk 0.3 — Accounts (do this now, some approvals take days)

- [Copernicus Data Space Ecosystem](#) account — for Sentinel-1/2 and Copernicus DEM.
- [Climate Data Store \(CDS\)](#) account — for ERA5. Get your API key immediately after signup (Settings → API key), you'll need it for scripted downloads.
- [Google Earth Engine](#) account (sign up, request access — usually fast) — huge time-saver for satellite data, optional but recommended.
- [Bhuvan \(ISRO\)](#) account — for Indian land-use/land-cover and some boundary data.
- Check **IMD data access** now (see Chunk 2.4) — this is the one that can have delays, so start early.

✅ **Checkpoint:** you can run `import geopandas, rasterio, xarray, torch` in a notebook with no errors, and you're logged into all four accounts above.

PHASE 1 — Define Your Study Area (Days 4-6)

Goal: one GeoJSON file that defines exactly where your project happens. Everything downloads and clips to this.

Chunk 1.1 — Get administrative boundaries

- Download **India district boundaries** shapefile from [Bhuvan](#) (Thematic Services → Administrative Boundaries) or from [GADM](#) (choose India, level 2 = district).

- Size: a few MB.

Chunk 1.2 — Build your polygon

- In QGIS or GeoPandas, select districts: Pune, Ahilyanagar (Ahmednagar), Solapur, Satara.
- You only want the **eastern parts** of Pune/Satara (the plain zone, not Western Ghats) — for a beginner, it's fine to start with full districts and refine the boundary later once you understand the terrain (Phase 2 DEM will show you where the hills are).
- Save as `study_area.geojson`.

✓ **Checkpoint:** open `study_area.geojson` in QGIS, it renders correctly over a basemap, area looks right (should be a few thousand km²).

Chunk 1.3 — Build the 250m grid

- Using GeoPandas, generate a regular 250m × 250m fishnet grid clipped to `study_area.geojson`.
- Assign each cell a unique `grid_id` (e.g., `MH_0001`, `MH_0002`...).
- Save as `grid_250m.geojson` / `grid_250m.parquet`.

✓ **Checkpoint:** grid renders in QGIS as a clean mesh inside your polygon. Count the cells — for a few thousand km² at 250m you'll have roughly 40,000–80,000 cells. That number matters: it tells you your data volume for every later step.

PHASE 2 — Download Core Datasets (Week 2–3)

Do these in order. Each one is a separate chunk — finish and validate one before starting the next.

Chunk 2.1 — DEM (terrain)

- Source: **Copernicus DEM GLO-30** via [Copernicus Data Space](#) or AWS Open Data (`s3://copernicus-dem-30m`), or **SRTM 30m** via [USGS EarthExplorer](#).
- Download only tiles covering your study area (not all of India).
- Size: tens of MB for your region.
- Process: clip to `study_area.geojson`, compute slope/aspect/curvature using `richdem` or QGIS terrain tools, aggregate to your 250m grid (mean elevation, mean slope, etc. per cell).

✓ **Checkpoint:** you have `grid_250m` with columns `elevation`, `slope`, `aspect` filled in, and a QGIS map showing elevation clearly varies across your region (hills near Satara/Pune west edge, flatter toward Solapur).

Chunk 2.2 — Soil (SoilGrids)

- Source: **ISRIC SoilGrids 250m** — use the bulk download/WCS route (`https://files.isric.org/soilgrids/latest/data/`), not the live REST API (it's flaky).
- Variables to get: clay, sand, organic carbon, bulk density, pH — just the **0–5cm** and **5–15cm** depths for now.
- Size: a few hundred MB after clipping to your region (SoilGrids is global — always clip immediately, don't download whole-world files).

✓ **Checkpoint:** grid has `soil_clay`, `soil_sand`, `soil_oc`, `soil_bd`, `soil_ph` columns, no major gaps (SoilGrids has near-full coverage, so gaps usually mean a clipping bug).

Chunk 2.3 — ERA5 (atmospheric reanalysis)

- Source: **CDS** — dataset name `reanalysis-era5-single-levels` (hourly), via the `cdsapi` Python package with your API key from Chunk 0.3.

- Variables (start small): 2m temperature, 2m dewpoint, surface pressure, 10m u/v wind, total precipitation.
- Time range: **start with 2018-2024** (not 25 years) — this alone is a meaningful download.
- Note: CDS requests are *queued*, not instant. Submit a request and expect to wait minutes to hours depending on load — don't panic if it doesn't return immediately.
- Size: for a small bounding box, hourly, 6 variables, ~6 years → a few GB in NetCDF/GRIB, less once converted to Parquet/Zarr.

✅ **Checkpoint:** you can load one month of ERA5 with `xarray`, plot temperature over your bounding box, and it looks physically sane (not all zeros, not NaN everywhere).

Chunk 2.4 — IMD (Indian ground truth)

- Source: **IMD** gridded rainfall/temperature products via [IMD Pune's data portal](#) or NDC (National Data Centre) — note some products require registration and a formal request; start this early since approval can take time.
- If access is slow: use a public **GHCN** or **APHRODITE** rainfall dataset as a stand-in for now, and swap in official IMD once access clears. Don't let this block Phase 3-5.
- Keep this dataset **separate** — you will use it mainly for **validation**, not training, so don't merge it into your main feature table yet.

✅ **Checkpoint:** you have at least one IMD (or substitute) rainfall time series for a handful of points in your study area, kept in its own folder, untouched by the training pipeline.

Chunk 2.5 — Sentinel-2 (vegetation/land surface)

- Source: **Copernicus Data Space** or **Google Earth Engine** (GEE is much easier for a beginner — you can compute NDVI/EVI composites server-side without downloading raw imagery).
- Start with **monthly cloud-free composites**, not every scene.
- Compute NDVI, EVI, NDWI per composite, aggregate to your 250m grid (mean/std per cell per month).

✅ **Checkpoint:** grid has monthly `ndvi_mean`, `ndvi_std` etc., and mapping NDVI in QGIS shows clear contrast between irrigated cropland and dry areas — this is a good sanity check that your pipeline is working.

Chunk 2.6 — Sentinel-1 (optional at this stage)

- Same source as above (GEE is easiest: VV/VH backscatter, monthly).
- This can wait until Phase 5 if you're short on time — it's a refinement, not a blocker.

✅ **Phase 2 overall checkpoint:** one table, keyed by `grid_id` (+ `month` where relevant), with terrain, soil, and vegetation columns filled in for every cell, plus a separate ERA5 time-series table keyed by `grid_id + timestamp`. This is your data foundation — don't proceed until this is clean.

PHASE 3 — Build the Baselines (Week 4)

Goal: before any "real" model, prove you have something to beat.

Chunk 3.1 — Climatology baseline

- For each grid cell, compute the historical average value (e.g., average July rainfall) and use that as your "prediction."

Chunk 3.2 — Persistence baseline

- Predict tomorrow = today (or this month = last month).

Chunk 3.3 — Evaluation setup

- Pick ONE target to start: **24h rainfall** (highest priority, hardest, most useful).
- Define your metrics now: MAE, RMSE.
- Define your **spatial holdout**: train on Pune + Ahilyanagar, test on Solapur/Satara — this must be decided before any model training, not after.

✅ **Checkpoint:** a small table showing Climatology MAE and Persistence MAE on your holdout region. These two numbers are your entire project's baseline to beat.

PHASE 4 — Engine 2 First: Historical/Temporal Model (Weeks 5-6)

Build this one first — it's simpler for a beginner (tabular ML, no graph theory needed).

Chunk 4.1 — Feature table

- For each `(grid_id + timestamp)`, create lag features: rainfall/temp/humidity at t-1, t-7, t-14, t-30 days, plus month/season indicators.

Chunk 4.2 — XGBoost model

- Train `(XGBoost)` or `(LightGBM)` regressor on this feature table, target = 24h rainfall.
- Evaluate on your spatial holdout from Phase 3.

✅ **Checkpoint:** XGBoost beats both baselines on MAE for at least the wetter season. If it doesn't beat climatology, don't move forward — debug features first (this is a very common and very fixable beginner issue, usually a data leakage or missing-lag bug).

Chunk 4.3 — Add LSTM (only after 4.2 works)

- Reshape data into sequences (e.g., 30-day windows) and train a small LSTM (2 layers, 64-128 hidden units) in PyTorch.
- Compare against XGBoost — it's fine (and common) if XGBoost wins at this stage; note it and move on.

✅ **Checkpoint:** you have `(P2 (prediction))` and, from an ensemble of 5 XGBoost models with different seeds, a rough uncertainty estimate `(U2)` (spread across the 5 predictions) for every grid cell + timestamp.

PHASE 5 — Engine 1: Physics/Spatial Model (Weeks 7-8)

Chunk 5.1 — Build the graph

- Nodes = your 250m grid cells (from Phase 1).
- Edges = connect each cell to its neighbors (simple 8-neighbor adjacency to start — don't add wind-direction/hydrology edges yet, that's a refinement).
- Node features = elevation, slope, soil, NDVI, current ERA5 state.

Chunk 5.2 — Small prototype first

- Before running this on your full 40,000+ cell grid, build a tiny subgraph (e.g., 500 cells) and get a `(GraphSAGE)` model (via PyTorch Geometric) training end-to-end on it. This proves the pipeline works before you burn hours on the full region.

Chunk 5.3 — Scale up

- Move to the full grid using PyTorch Geometric's `(NeighborLoader)` (don't try to load the whole graph onto a 4GB GPU at once — this will crash).
- Train GraphSAGE to predict the same target (24h rainfall) against the same holdout split as Engine 2.

✅ **Checkpoint:** GraphSAGE trains without crashing, and its MAE on the holdout is in the same ballpark as XGBoost (better or worse is fine — what matters is that it's a real, working, independent prediction, `(P1)` + an uncertainty estimate `(U1)` from a small ensemble).

PHASE 6 — Fusion (Week 9-10)

Chunk 6.1 — Check independence

- Before fusing, compute the correlation between Engine 1's errors and Engine 2's errors on the holdout set. If they're highly correlated, fusion won't help much — worth knowing now, not after.

Chunk 6.2 — Fusion methods, in order

1. Simple 50/50 average — trivial baseline.
2. Validation-weighted average (weight by inverse MAE from Phase 3-5 holdout results).
3. Uncertainty-weighted (inverse-variance using σ_1 , σ_2).

Chunk 6.3 — Compare all five models in one table

Climatology / Persistence / XGBoost alone / GraphSAGE alone / Fusion — on the same holdout, same metric.

✅ **Checkpoint:** you have a real table showing whether fusion actually helps. This table is the core result of your project — whatever it shows, it's a valid finding.

PHASE 7 — Validation & Dashboard (Weeks 11-12)

Chunk 7.1 — Validate against IMD

- Compare your fused prediction against the IMD data you set aside in Chunk 2.4 — this is your most credible validation since IMD was never used in training.

Chunk 7.2 — Simple dashboard

- A basic map (Leaflet/MapLibre or even just a Streamlit app) showing: select a village → see predicted rainfall/temp, confidence, and physics-vs-historical contribution.
- Don't build the full Next.js frontend from the PRD yet — a Streamlit prototype is enough to demo and iterate on.

✅ **Final checkpoint for this phase:** you can click any village in your study area and see a prediction with an uncertainty band, validated against held-out IMD data.

What NOT to touch until the above works

Crop impact modules, SHAP/LLM advisory text, IoT sensors, WhatsApp/crowdsourcing, pest/disease risk, yield prediction, hail/flood modules, the "foundation model" idea. All of these sit on top of a working, validated climate engine — building them earlier just produces more surface area with no validated core underneath it.

Quick reference — what to download, in order

Phase	Dataset	Source	Approx size (your region)
1	District boundaries	Bhuvan / GADM	few MB
2.1	DEM	Copernicus DEM GLO-30 / SRTM	tens of MB
2.2	Soil	ISRIC SoilGrids (bulk download)	~hundreds of MB
2.3	ERA5 (2018–2024)	CDS via <code>cdsapi</code>	few GB
2.4	IMD rainfall/temp	IMD Pune / NDC (or GHCN as stand-in)	small
2.5	Sentinel-2 composites	Copernicus Data Space / GEE	manageable via GEE, avoid raw downloads
2.6	Sentinel-1 (optional)	GEE	manageable via GEE